

# Adversarial Co-Evolution of RL and LLM Agents in Gin Rummy: A Phase-1/2 System Report

Nima Kelidari   Mahdi Salmani   Mohammadsaeed Haghi  
{kelidari, salmanis, haghim}@usc.edu  
University of Southern California

## Abstract

We build a hybrid system in which a lightweight action-masked PPO agent and a strong-but-slow LLM opponent train against each other in Gin Rummy, without paying full LLM inference latency on every reinforcement-learning step. This report covers two phases. In **Phase 1** an action-masked PPO agent saturates against a random opponent: an eight-configuration hyperparameter sweep lands every run in a 98.3%–99.6% win-rate band, statistically indistinguishable at 1000-episode evaluation. Because further tuning against weak opponents is unproductive, **Phase 2** builds the infrastructure for LLM-in-the-loop training: a master/worker/cache inference stack with shared-filesystem service discovery and a suit-symmetry prompt cache, plus self-play and AlphaZero-style pool curricula for an in-distribution opponent. We report three operational findings: (i) OLMoE-1B is too weak to teach (it plays at chance), whereas Qwen2.5-7B-Instruct beats a random hero 5/5 and is a genuinely useful teacher; (ii) loading a 7B worker from the home NFS is  $62\times$  slower than from scratch/BeeGFS and must be avoided; and (iii) pool self-play diverged after  $\sim 10\text{M}$  steps, dropping its win rate vs random from 98.5% to 85.8%. Finally we run a full **1.5M-step RL-vs-LLM fine-tune** (PPO warm-started from the self-play champion, up to 32 Qwen2.5-7B workers): episode reward stays flat at the knock value and the resulting agent matches—but does not beat—its progenitor. The honest conclusion is that a 7B opponent is *weaker* than our self-play champion, so it cannot lift a near-saturated policy; making the LLM a stronger or co-evolving teacher is the Phase-3 problem.

## 1 Goal and roadmap

Gin Rummy is an imperfect-information card game requiring both short-horizon arithmetic (deadwood counting) and long-horizon planning (meld formation). Our roadmap has three phases: (1) train a strong RL backbone against weak opponents; (2) train that backbone against an LLM opponent that is strategically richer but operationally far slower; (3) let the two co-evolve. We use action-masked PPO [4, 1] on the PettingZoo [5] `gin_rummy_v4` environment (wrapping RLCard [6]) with Stable-Baselines3 [3]; masking sets illegal-action logits to  $-\infty$  inside a custom actor-critic policy. We first fixed an upstream bug in which PettingZoo silently reverts the configured `knock/gin` rewards on every seeded reset (PettingZoo PR #1312); all results below use the corrected rewards (`knock` = 0.5, `gin` = 1.5).

## 2 Phase 1: the RL backbone saturates against random

We sweep eight configurations: a  $3\times 2$  grid over learning rate  $\{3\cdot 10^{-4}, 10^{-4}, 5\cdot 10^{-5}\}$  and entropy coefficient  $\{0.01, 0.03\}$ , plus two ablations (rollout length 1024; 10 epochs). Each trains for  $2\times 10^6$  steps with 96 parallel envs on a 64-core node and is evaluated over 1000 deterministic games vs `RandomAgent`. Every configuration converges to the 98.3%–99.6% band (Fig. 1, Table 1); the spread is about one binomial 95% CI ( $\pm 0.6$  pp at  $p=0.99$ ), so the configurations cannot be separated. The shared failure mode is informative: mean episode reward clusters at the knock value (0.5) rather than the gin value (1.5) (Fig. 1b), i.e. the agent takes the certain knock instead of holding for a gin—a risk/reward decision that only a *thinking* opponent can teach. This is what motivates Phase 2.

## 3 Phase 2: infrastructure for LLM-in-the-loop training

A single PPO rollout fires up to  $\sim 50\text{k}$  opponent queries; at 0.5–3 s per 7B call, a naïve loop takes hours. We decouple inference from RL with a three-tier stack. **Workers** (one GPU each, HF backend) load the model and register themselves in a shared-filesystem registry. The **master** (CPU) discovers live workers, load-balances, and owns a prompt cache. The RL **client** (96 env subprocesses) reaches the master through the existing Ollama-compatible API, so no RL code changes. We chose the HuggingFace backend because the installed vLLM [2] 0.8.5 is incompatible with transformers 5.7. The cache uses a *suit-symmetry canonical key*: Gin Rummy is invariant under suit relabeling, so suit-equivalent states share one entry and the chosen action is de-canonicalized on return.

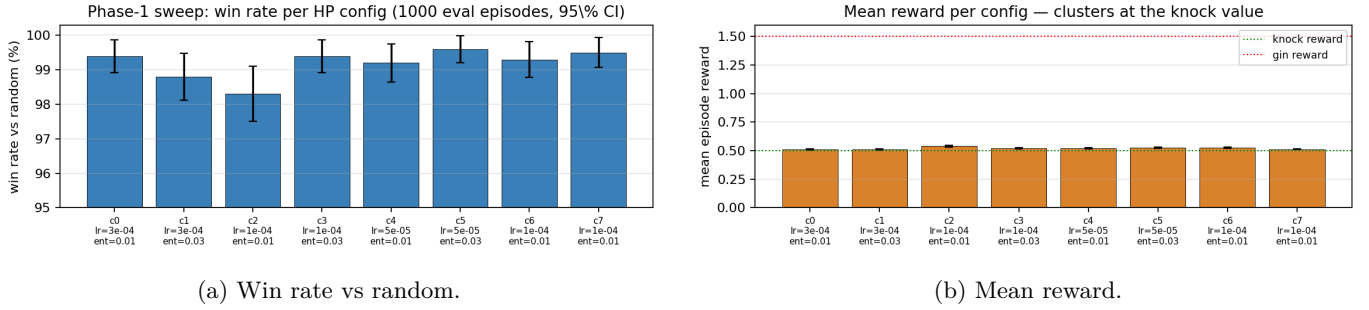


Figure 1: Phase-1 sweep. All eight configs saturate vs random (a); reward sits at the knock value, far below gin (b), exposing the improvement headroom.

Table 1: Phase-1 leaderboard (top rows) and Phase-2 opponent panel (bottom). Win/loss over 1000 deterministic eval games unless noted.

| Agent / config                  | opponent         | win%         | loss% | note                    |
|---------------------------------|------------------|--------------|-------|-------------------------|
| PPO cfg5 (lr 5e-5, ent .03)     | random           | 99.6         | 0.4   | best of sweep           |
| PPO cfg7 (lr 1e-4, 10 epochs)   | random           | 99.5         | 0.5   | ablation                |
| PPO cfg0 (lr 3e-4, ent .01)     | random           | 99.4         | 0.6   | baseline                |
| PPO cfg3 (lr 1e-4, ent .03)     | random           | 99.4         | 0.6   |                         |
| PPO cfg6 (lr 1e-4, $n_s=1024$ ) | random           | 99.3         | 0.7   | ablation                |
| PPO cfg4 (lr 5e-5, ent .01)     | random           | 99.2         | 0.8   |                         |
| PPO cfg1 (lr 3e-4, ent .03)     | random           | 98.8         | 1.2   |                         |
| PPO cfg2 (lr 1e-4, ent .01)     | random           | 98.3         | 1.7   | worst of sweep          |
| Self-play (3M, vs frozen run_5) | run_5            | 61.1         | 38.9  | beats progenitor        |
| Self-play (3M)                  | random           | 98.7         | 1.3   | stays dominant          |
| Pool champion (12M)             | random           | 85.8         | 14.2  | <i>diverged</i>         |
| Pool champion (12M)             | run_5            | 62.0         | 38.0  |                         |
| OLMoE-1B opponent               | random           | $\approx 50$ | —     | plays at chance         |
| Qwen2.5-7B opponent             | random           | <b>100</b>   | 0     | 5/5 (small $N$ )        |
| LLM-finetuned PPO (1.5M)        | random           | 99.0         | 1.0   | retains competence      |
| LLM-finetuned PPO (1.5M)        | self-play champ  | 43.3         | 56.7  | <i>no net gain</i>      |
| LLM-finetuned PPO (1.5M)        | winrate / reward | 55.3 / 61.3  | —     | still beats weak agents |

**Two infrastructure findings.** First, loading a 7B worker from the home NFS runs at  $\sim 11$  MB/s and takes  $\sim 28$  min—long enough to blow the health-check timeout. Staging weights on scratch/BeeGFS cuts this to 27 s, a  $62\times$  speedup (Fig. 3c); this is mandatory when launching dozens of workers. Second, the in-game opponent model is selected purely by the worker’s loaded weights—the master ignores the client’s model string—so swapping opponents is a one-line preset change.

## 4 Opponents: self-play, pool, and the LLM

**Self-play.** Fine-tuning the Phase-1 champion against a *frozen* copy of itself for 3M steps yields 58–61% vs that progenitor while holding  $\sim 98.7\%$  vs random (Fig. 3a)—one clean generation of improvement.

**Pool (AlphaZero-style).** Training against a growing pool of past checkpoints ran to 12M steps but *diverged* after  $\sim 10$ M: vs-random win rate fell from  $\sim 98.5\%$  to 85.8% (Fig. 3b). We keep the pre-divergence checkpoints; the production opponent reverts to self-play.

**LLM opponent.** OLMoE-1B cannot follow the action-format prompt—most responses fail to parse and fall back to a random move, so it plays at chance and cannot teach. Qwen2.5-7B-Instruct, by contrast, beats a random hero 5/5 (Fig. 3d): a genuinely competent teacher. Its chain-of-thought prompt costs  $\sim 16$ s/call (1024 output tokens); for training we switch to a terse prompt that emits only the action and cap output tokens, cutting latency by an order of magnitude.

**Human play.** A zero-dependency web client (HTML/JS, Fig. 2) lets a human play any of the four trained opponents (Phase-1, self-play, pool, reward-shaped) with 3-D card animations; it reads opponent state from RLCard and was validated in headless Chromium.



Figure 2: The human-vs-RL web client (debug view, opponent hand revealed). The player drafts and discards against any trained agent; the board animates draws, reveals, and discards. The same LLMagent path that the RL loop uses also drives an optional LLM opponent here.

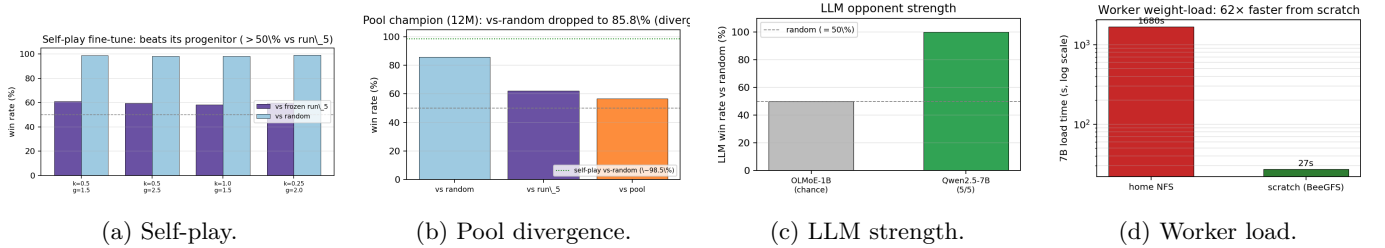


Figure 3: Phase-2 results. (a) self-play beats its progenitor; (b) the pool champion regressed vs random after divergence; (c) Qwen2.5-7B is a competent opponent where OLMoE-1B is not; (d) scratch staging is 62 $\times$  faster.

## 5 Full RL-vs-LLM run: results, limitations, next steps

We then ran the real experiment: PPO warm-started from the self-play champion, 96 env subprocesses, fine-tuned for **1.5M steps** against the master + a pool of up to 32 Qwen2.5-7B workers (the scheduler granted  $\sim 14$  under the shared GPU quota; the pool is elastic). The serving stack is the headline infrastructure win: the terse prompt + a 64-token cap cut effective per-call latency from  $\sim 16$ s to  $\sim 0.4$ s, the master sustained  $\sim 32$  load-balanced queries/s ( $\sim 500\times$  a single-stream chain-of-thought loop), and PPO held  $\sim 39$  env-steps/s—so a rollout completes in minutes and the whole run finished in a few hours.

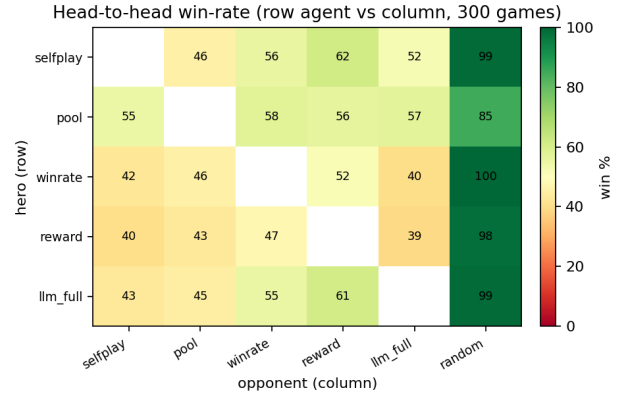
**The learning result is negative and clean.** Across all 1.5M steps the mean episode reward stays pinned at the knock value  $\approx 0.49$  and never trends toward gin (1.5) (Fig. 4a). The fine-tuned agent *retains* 99.0% vs random and still beats the weaker reward-shaped agents, but sits at 43.3% vs the self-play champion it started from (Fig. 4b, Table 1) —no net gain. The cause is structural: Qwen2.5-7B with a terse prompt is a *weaker* player than our self-play champion, so it provides no gradient toward stronger play; training a near-saturated policy against a weaker teacher holds it in place (and slightly perturbs it). The suit-symmetry cache also stayed near 0% hit-rate—opening states are too diverse to recur at this scale—so throughput is purely worker-bound.

**Limitations.** (i) The teacher is weaker than the trainee—the central reason no improvement appears; (ii) the terse prompt trades opponent quality for throughput, and we have not measured a strong-prompt / larger-model teacher; (iii) single seed per run and a small ( $N=5$ ) LLM-strength sample; (iv) the prompt cache yields no speedup at current volume; (v) pool self-play diverged past 10M; (vi) the midterm’s negative Dagger and dense-reward-shaping results remain code-unaudited. Phase 3 (below) directly addresses (i)/(ii) with a stronger Qwen3-30B teacher and (iii) with a three-seed reward-shaping sweep.

**Phase 3 (in progress).** Two experiments are now running on the cluster; results will be folded into this report as they land. **(A) Closing the gin headroom:** since Phase 1’s agent always knocks and never gins, we sweep six reward configs (gin incentive escalating from baseline to gin-only)  $\times$  three seeds, fine-tuning the champion for 8M self-play steps each, with a periodic gin-rate eval every 1M steps that yields a gin-rate-vs-steps curve and keeps the best-by-gin-rate checkpoint (so a longer run cannot be hurt by the late divergence we saw in pool play); the decisive metric is the gin rate scored under *fixed* eval rewards, comparable across configs. **(B) A stronger teacher:** we test whether Qwen3-30B beats the self-play champion—if so it becomes the teacher for a fine-tune (the real co-evolution payoff); if a 30B still loses to a 12M-step specialist, that bounds the LLM-teacher thesis and (A) carries the result. Further out: offline CQL distillation of bulk LLM-vs-LLM games, and a strong- vs. terse-prompt teacher ablation.



(a) Reward flat over 1.5M steps.



(b) Head-to-head win-rate.

Figure 4: Full RL-vs-LLM run. (a) episode reward never leaves the knock value, so the LLM teacher did not raise the gin rate; (b) the fine-tuned agent (`llm_full`) lands at champion level—beating the weak agents and random, roughly even vs the champion it came from.

**Reproducibility.** Sweeps run as SLURM arrays under `sweep/`; the LLM stack is three `slurm/` jobs — master, a worker array (scratch-staged weights), and the trainer; every figure here is regenerated by `make_figures.py` from the JSON result files.

## References

- [1] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. *arXiv preprint arXiv:2006.14171*, 2022.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. 2023.
- [3] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. In *JMLR*, 2021.
- [4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [5] J. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Caroline Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. Pettingzoo: Gym for multi-agent reinforcement learning. *NeurIPS*, 2021.
- [6] Daochen Zha, Kwei-Herng Lai, Yuanpu Cao, Songyi Huang, Ruzhe Wei, Junyu Guo, and Xia Hu. Rlcard: A platform for reinforcement learning in card games. *IJCAI Workshop*, 2020.